

Insurance Claims Processing System

File: Configurations.java

1. **Variables:** None.
2. **Constructors:** None (Spring-managed @Configuration class).
3. **Method Signatures:**
 - a. `public PasswordEncoder passwordEncoder()` → Returns BCryptPasswordEncoder for password hashing.
 - b. `public WebMvcConfigurer corsConfigurer()` → Configures CORS to allow all origins and specified HTTP methods.

File : SecurityConfig.Java

- **Variables:**
 - `userDetailsService`: Manages user authentication.
 - `jwtRequestFilter`: Handles JWT authentication.
 - `passwordEncoder`: Encrypts user passwords.
- **Constructor:**
 - `public SecurityConfig(...)` → Initializes `userDetailsService`, `jwtRequestFilter`, and `passwordEncoder`.
- **Method Signatures:**
 1. `protected void configure(AuthenticationManagerBuilder auth)` → Configures authentication with `UserDetailsService` and `passwordEncoder`.
 2. `protected void configure(HttpSecurity http)` → Defines security rules, role-based access, and disables session management (stateless authentication using JWT).
 - `POST /api/user/register` → User registration.
 - `POST /api/user/login` → User login.
 - `PUT /api/adjuster/claim/{id}` → Update claim.
 - `GET /api/adjuster/claims` → View all claims.
 - `GET /api/adjuster/underwriters` → View underwriters.
 - `PUT /api/adjuster/claim/{claimId}/assign` → Assign claim.
 - `POST /api/policyholder/claim` → Submit claim.
 - `GET /api/policyholder/claims` → View own claims.
 - `POST /api/investigator/investigation` → Create investigation.
 - `PUT /api/investigator/investigation/{id}` → Update investigation.
 - `GET /api/investigator/investigations` → View investigations.
 - `PUT /api/underwriter/claim/{id}/review` → Review claim.
 - `GET /api/underwriter/claims` → View claims.
 3. `public AuthenticationManager authenticationManagerBean()` → Exposes `AuthenticationManager` as a Spring bean.

File: AdjusterController.java

- **Variables:**
 - @Autowired private UnderwriterRepository underwriterRepository: Repository for managing underwriters.
 - @Autowired private ClaimService claimService: Service handling claim-related operations.
- **Constructor:**
 - No explicit constructor (Spring-managed with @Autowired).
- **Method Signatures:**
- @PutMapping("/api/adjuster/claim/{id}")
public ResponseEntity<Claim> updateClaim(@PathVariable Long id, @RequestBody Claim claimDetails)
 - Updates a claim based on its ID.
- @GetMapping("/api/adjuster/claims")
public List<Claim> getAllClaims()
 - Retrieves all claims.
- @GetMapping("/api/adjuster/underwriters")
public List<Underwriter> getAllUnderwriters()
 - Retrieves all underwriters.
- @PutMapping("/api/adjuster/claim/{claimId}/assign")
public ResponseEntity<Claim> assignClaimToUnderwriter(@PathVariable Long claimId, @RequestParam Long underwriterId)
 - Assigns a claim to an underwriter.

File : InvestigatorController.java

- **Variables:**
 - @Autowired private InvestigationService investigationService: Handles investigation-related operations.
- **Constructor:**
 - No explicit constructor (Spring-managed with @Autowired).
- **Method Signatures:**
- @PostMapping("/api/investigator/investigation")
public ResponseEntity<Investigation> createInvestigation(@RequestBody Investigation investigation)
 - Creates a new investigation.
- @PutMapping("/api/investigator/investigation/{id}")
public ResponseEntity<Investigation> updateInvestigation(@PathVariable Long id, @RequestBody Investigation investigationDetails)
 - Updates an existing investigation based on its ID.
- @GetMapping("/api/investigator/investigations")
public List<Investigation> getAllInvestigations()
 - Retrieves all investigations.

File : PolicyholderController.java

□ **Variables:**

- @Autowired private ClaimService claimService: Service for managing claim-related operations.

□ **Method Signatures:**

- @PostMapping("/api/policyholder/claim")
public ResponseEntity<Claim> submitClaim(@RequestParam Long policyholderId, @RequestBody Claim claim)
 - Submits a new claim for a policyholder.
- @GetMapping("/api/policyholder/claims")
public ResponseEntity<List<Claim>> getClaims(@RequestParam Long policyholderId)
 - Retrieves all claims submitted by a specific policyholder.

File : RegisterAndLoginController

• **Variables:**

- @Autowired private UserService userService: Handles user-related operations.
- @Autowired private JwtUtil jwtUtil: Utility for generating JWT tokens.
- @Autowired private AuthenticationManager authenticationManager: Manages authentication.

• **Method Signatures:**

- @PostMapping("/api/user/register")
public ResponseEntity<User> registerUser(@RequestBody User user)
 - Registers a new user and returns the created user object with 201 Created status.
- @PostMapping("/api/user/login")
public ResponseEntity<LoginResponse> loginUser(@RequestBody LoginRequest loginRequest)
 - **Description:** Authenticates the user, generates a JWT token upon successful login, and returns the token along with user details.
 - **Response:** Returns 200 OK with a LoginResponse object containing user ID, token, username, email, and role. If authentication fails, it throws a 401 Unauthorized error.

File : UnderwriterController

Variables:

- @Autowired private ClaimService claimService → Manages claim-related operations.

Method Signatures:

- @PutMapping("/api/underwriter/claim/{id}/review")
public ResponseEntity<Claim> reviewClaim(@PathVariable Long id, @RequestParam String status)
 - **Description:** Updates the status of a claim (e.g., Approved, Rejected).
 - **Response:** Returns 200 OK with the updated Claim object.
- @GetMapping("/api/underwriter/claims")
public ResponseEntity<List<Claim>> getClaimsForReview(@RequestParam Long underwriterId)

- **Description:** Retrieves claims assigned to an underwriter for review.
- **Response:** Returns 200 OK with a list of Claim objects.

File : LoginRequest.java

Fields:

- private String username → Stores the username for authentication.
- private String password → Stores the password for authentication.

Constructor:

- @JsonCreator

public LoginRequest(@JsonProperty("username") String username, @JsonProperty("password") String password)

- **Description:** Initializes LoginRequest with username and password.

Methods:

- public String getUsername() → Returns the username.
- public void setUsername(String username) → Sets the username.
- public String getPassword() → Returns the password.
- public void setPassword(String password) → Sets the password.

File : LoginResponse.java

Fields:

- private Long userId → Stores the user's ID.
- private String token → Stores the generated JWT token.
- private String username → Stores the username.
- private String email → Stores the email.
- private String role → Stores the user role.

Constructor:

- @JsonCreator

public LoginResponse(@JsonProperty("userId") Long userId, @JsonProperty("token") String token, @JsonProperty("username") String username, @JsonProperty("email") String email, @JsonProperty("role") String role)

- **Description:** Initializes LoginResponse with user details and JWT token.

Methods:

- public Long getUserId() → Returns the user ID.
- public void setUserId(Long userId) → Sets the user ID.
- public String getToken() → Returns the JWT token.
- public void setToken(String token) → Sets the JWT token.
- public String getUsername() → Returns the username.
- public void setUsername(String username) → Sets the username.
- public String getEmail() → Returns the email.
- public void setEmail(String email) → Sets the email.
- public String getRole() → Returns the role.

- public void setRole(String role) → Sets the role.

File : Adjuster.java

Class:

- public class Adjuster extends User → Represents an insurance adjuster, inheriting from the User entity.

Annotations:

- @Entity → Marks this class as a JPA entity for database persistence.

Description:

- Extends the User class to include additional attributes and functionalities specific to insurance adjusters.

File : Claim.java

Class:

- public class Claim → Represents an insurance claim submitted by a policyholder.

Annotations:

- @Entity → Marks this class as a JPA entity.
- @Id @GeneratedValue(strategy = GenerationType.IDENTITY) → Defines id as the primary key with auto-increment.

Fields:

- private Long id → Unique identifier for the claim.
- private String description → Details of the claim.
- private Date date → Date when the claim was submitted.
- private String status → Current status of the claim.

Relationships:

- @ManyToOne @JoinColumn(name = "policyholder_id") private Policyholder policyholder → The policyholder who submitted the claim.
- @ManyToOne @JoinColumn(name = "adjuster_id") private Adjuster adjuster → The adjuster reviewing the claim.
- @ManyToOne @JoinColumn(name = "underwriter_id") private Underwriter underwriter → The underwriter assessing the claim.
- @OneToOne(mappedBy = "claim") @JsonIgnore private Investigation investigation → The investigation related to this claim.

Method Signatures:

- public Long getId() → Returns the claim ID.
- public void setId(Long id) → Sets the claim ID.
- public String getDescription() → Returns the claim description.
- public void setDescription(String description) → Sets the claim description.
- public Date getDate() → Returns the claim date.
- public void setDate(Date date) → Sets the claim date.
- public String getStatus() → Returns the claim status.
- public void setStatus(String status) → Sets the claim status.

- `public Policyholder getPolicyholder()` → Returns the policyholder.
- `public void setPolicyholder(Policyholder policyholder)` → Sets the policyholder.
- `public Adjuster getAdjuster()` → Returns the adjuster.
- `public void setAdjuster(Adjuster adjuster)` → Sets the adjuster.
- `public Underwriter getUnderwriter()` → Returns the underwriter.
- `public void setUnderwriter(Underwriter underwriter)` → Sets the underwriter.
- `public Investigation getInvestigation()` → Returns the investigation.
- `public void setInvestigation(Investigation investigation)` → Sets the investigation.

File : Investigation.java

Class:

- `public class Investigation` → Represents an investigation conducted on an insurance claim.

Annotations:

- `@Entity` → Defines this as a JPA entity for ORM mapping.
- `@Id @GeneratedValue(strategy = GenerationType.IDENTITY)` → Defines a unique, auto-generated primary key for each investigation.

Fields:

- `private Long id` → Unique identifier for the investigation.
- `private String report` → Details of the investigation report.
- `private String status` → Current status of the investigation (e.g., "Pending", "Completed").

Relationships:

- `@OneToOne @JoinColumn(name = "claim_id") private Claim claim;`
 - Links the investigation to a specific claim.
 - Ensures each claim can have only one corresponding investigation.

Methods:

- `public Long getId()` → Returns the investigation ID.
- `public void setId(Long id)` → Sets the investigation ID.
- `public String getReport()` → Retrieves the investigation report.
- `public void setReport(String report)` → Updates the report content.
- `public String getStatus()` → Gets the current investigation status.
- `public void setStatus(String status)` → Updates the investigation status.
- `public Claim getClaim()` → Retrieves the associated claim.
- `public void setClaim(Claim claim)` → Links an investigation to a claim.

File : Investigator.java

Class:

- `public class Investigator` → Represents an investigator responsible for handling insurance claim investigations.

Annotations:

- `@Entity` → Defines this as a JPA entity for ORM mapping.

Inheritance:

- `extends User` → Inherits common user attributes and behavior from the User entity.

Additional Fields:

- Contains specific fields and methods relevant to investigators (not defined yet).

File : Policyholder.Java

Class:

- public class Policyholder → Represents a policyholder who owns insurance policies and can file claims.

Annotations:

- @Entity → Marks this class as a JPA entity for database mapping.

Inheritance:

- extends User → Inherits user-related attributes and behavior from the User entity.

Additional Fields:

- Contains policyholder-specific attributes and methods

File : Underwriter.java

Class:

- public class Underwriter → Represents an underwriter responsible for evaluating and approving insurance claims.

Annotations:

- @Entity → Marks this class as a JPA entity for database mapping.

Inheritance:

- extends User → Inherits user-related attributes and behavior from the User entity.

Additional Fields:

- Contains underwriter-specific attributes and methods

File : User.java

Class:

- public class User → Represents a base user in the insurance claims processing system. Other user types (e.g., Policyholder, Adjuster, Underwriter) extend this class.

Annotations:

- @Entity → Declares this as a JPA entity.
- @Table(name = "users") → Maps the entity to the users table.
- @Inheritance(strategy = InheritanceType.JOINED) → Uses joined table inheritance, allowing different user types to have their own specific tables while sharing common fields.

Fields:

- id (Long) → Auto-generated unique identifier for each user.
- username (String) → Unique username for authentication.
- password (String) → Stores the user's encrypted password.

- email (String) → Email address for communication and verification.
- role (String) → Specifies the user type (e.g., "Policyholder", "Adjuster", "Underwriter").

Methods:

- Getters and setters for all fields to manage user data.
- getId() / setId(Long id) → Retrieves or assigns the user ID.
- getUsername() / setUsername(String username) → Manages the username.
- getPassword() / setPassword(String password) → Handles password encryption and updates.
- getEmail() / setEmail(String email) → Retrieves or updates the email.
- getRole() / setRole(String role) → Assigns or retrieves the user's role.

This base class ensures that all users have common attributes while allowing specialized user roles to extend and add more functionalities as needed.

File : JwtRequestFilter.java

Class Summary:

JwtRequestFilter is a security filter that intercepts HTTP requests, extracts JWT tokens, validates them, and authenticates users in the Spring Security context.

Instance Variables:

- UserDetailsServiceImpl userDetailsService – Loads user details for authentication.
- JwtUtil jwtUtil – Handles JWT extraction and validation.

Constructor:

- JwtRequestFilter(UserDetailsServiceImpl userDetailsService, JwtUtil jwtUtil)
Initializes dependencies required for JWT authentication.

Methods:

- protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)
Extracts JWT from the request, validates it, retrieves user details, and sets authentication in the security context before proceeding with the request.

File : JwtUtil.java

Class Summary:

JwtUtil is a utility class responsible for generating, parsing, and validating JWT tokens for authentication and authorization in the application.

Instance Variables:

- UserRepository userRepository – Repository for fetching user details.
- String secret – Secret key for signing JWT tokens.
- int expiration – Token expiration time in seconds.

Constructor:

- JwtUtil(UserRepository userRepository)
Initializes the repository for user data retrieval.

Methods:

- public String generateToken(String username)
Generates a JWT token with user-specific claims, including role and expiration time.
- public Claims extractAllClaims(String token)
Extracts all claims from the provided JWT token.

- public String extractUsername(String token)
Retrieves the username from the JWT token.
- public boolean isTokenExpired(String token)
Checks if the JWT token is expired.
- public boolean validateToken(String token, UserDetails userDetails)
Validates the token by verifying the username and expiration status.

File : AdjusterRepository.java

AdjusterRepository is a Spring Data JPA repository for managing Adjuster entity persistence and database operations.

Extends:

- JpaRepository<Adjuster, Long> – Provides built-in CRUD operations for the Adjuster entity with Long as the primary key type.

Custom Query Methods:

- Additional query methods can be defined as needed using Spring Data JPA's method naming conventions or @Query annotations.

File : ClaimRepository.java

ClaimRepository is a Spring Data JPA repository for managing Claim entity persistence and database operations.

Extends:

- JpaRepository<Claim, Long> – Provides built-in CRUD operations for the Claim entity with Long as the primary key type.

Custom Query Methods:

- List<Claim> findByPolicyholder(Policyholder policyholder); – Retrieves all claims associated with a specific policyholder.
- List<Claim> findByUnderwriter(Underwriter underwriter); – Retrieves all claims assigned to a specific underwriter.

File : InvestigationRepository.java

InvestigationRepository is a Spring Data JPA repository for managing Investigation entity persistence and database operations.

Extends:

- JpaRepository<Investigation, Long> – Provides built-in CRUD operations for the Investigation entity with Long as the primary key type.

Custom Query Methods:

- Additional query methods can be defined as needed to retrieve investigations based on specific criteria.

File : InvestigatorRepository.java

InvestigatorRepository is a Spring Data JPA repository for managing Investigator entity persistence and database operations.

Extends:

- JpaRepository<Investigator, Long> – Provides built-in CRUD operations for the Investigator entity with Long as the primary key type.

Custom Query Methods:

- Additional query methods can be defined to retrieve investigators based on specific criteria.

File : PolicyholderRepository.java

PolicyholderRepository is a Spring Data JPA repository for managing Policyholder entity persistence and database operations.

Extends:

- JpaRepository<Policyholder, Long> – Provides built-in CRUD operations for the Policyholder entity with Long as the primary key type.

Custom Query Methods:

- Additional query methods can be defined to retrieve policyholders based on specific criteria.

File : UnderwriterRepository.java

UnderwriterRepository is a Spring Data JPA repository for managing Underwriter entity persistence and database operations.

Extends:

- JpaRepository<Underwriter, Long> – Provides built-in CRUD operations for the Underwriter entity with Long as the primary key type.

Custom Query Methods:

- Additional query methods can be defined to retrieve underwriters based on specific criteria.

File : UserRepository.java

UserRepository is a Spring Data JPA repository for managing User entity persistence and database operations.

Extends:

- JpaRepository<User, Long> – Provides built-in CRUD operations for the User entity with Long as the primary key type.

Custom Query Methods:

- User findByUsername(String username); – Retrieves a User entity by its username.

File : ClaimService.java

ClaimService is a Spring service that manages insurance claim operations, including creation, updates, retrieval, assignment, and review.

Instance Variables:

- ClaimRepository claimRepository – Handles CRUD operations for Claim.
- PolicyholderRepository policyholderRepository – Manages Policyholder entity interactions.
- UnderwriterRepository underwriterRepository – Manages Underwriter entity interactions.

Constructor:

- ClaimService() – Default constructor with @Autowired dependencies.

Method Signatures:

- public Claim createClaim(Claim claim) – Creates a new claim.
- public Claim updateClaim(Long id, Claim claimDetails) – Updates claim details by ID.
- public List<Claim> getAllClaims() – Retrieves all claims.
- public Claim submitClaim(Long policyholderId, Claim claim) – Submits a claim for a specific policyholder.
- public List<Claim> getClaimsByPolicyholder(Long policyholderId) – Fetches all claims associated with a policyholder.
- public Claim reviewClaim(Long id, String status) – Updates claim status during review.
- public List<Claim> getClaimsForReview(Long underwriterId) – Retrieves claims assigned to a specific underwriter.
- public Claim assignClaimToUnderwriter(Long claimId, Long underwriterId) – Assigns a claim to an underwriter.

File : InvestigationService.java

InvestigationService is a Spring service that manages insurance investigations, including creation, updates, and retrieval.

Instance Variable:

- InvestigationRepository investigationRepository – Handles CRUD operations for Investigation.

Constructor:

- InvestigationService() – Default constructor with @Autowired dependencies.

Method Signatures:

- public Investigation createInvestigation(Investigation investigation) – Creates a new investigation.
- public Investigation updateInvestigation(Long id, Investigation investigationDetails) – Updates an investigation's details by ID.
- public List<Investigation> getAllInvestigations() – Retrieves all investigations.

File : UserService.java

UserService is a Spring service implementing UserDetailsService for user authentication and management. It handles user registration, retrieval, and authentication.

Instance Variables:

- UserRepository userRepository – Manages user data persistence.
- AdjusterRepository adjusterRepository – Manages adjuster data.
- InvestigatorRepository investigatorRepository – Manages investigator data.
- PolicyholderRepository policyholderRepository – Manages policyholder data.
- UnderwriterRepository underwriterRepository – Manages underwriter data.
- PasswordEncoder passwordEncoder – Encodes passwords securely.

Constructor:

- UserService() – Default constructor with @Autowired dependencies.

Method Signatures:

- public User registerUser(User user) – Registers a user based on their role and saves them to the respective repository.
- public User getUserByUsername(String username) – Retrieves a user by their username.
- public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException – Loads user details for authentication.
- private void copyProperties(User source, User target) – Copies user details while encoding the password.

*****END
OF BACKEND*****
Front End –

File: AssignClaimComponent

Variables

- **itemForm:** FormGroup - Manages form controls for claim assignment.
- **formModel:** any - Stores initial form values (claimId defaulted to null).
- **showError:** boolean - Flags if an error message should be displayed.
- **errorMessage:** any - Stores the error message text.
- **assignModel:** any - Placeholder object for assignment data (not used explicitly).
- **showMessage:** any - Flags if a success message should be displayed.
- **responseMessage:** any - Stores success message text.
- **claimList:** any[] - Holds the list of claims fetched from the backend.
- **underwriterList:** any[] - Holds the list of underwriters fetched from the backend.

Constructor Signature: constructor(

```
public router: Router,  
public httpService: HttpService,  
private formBuilder: FormBuilder,  
private authService: AuthService  
)
```

1. onSubmit()

- **Purpose:** Submits the claim assignment form.
- **Variables:**
 - itemForm: FormGroup - Holds form controls for claim assignment.

- showError: boolean - Flags if an error message should be displayed.
- showMessage: boolean - Flags if a success message should be displayed.
- responseMessage: string - Stores success or error messages.
- **Functionality:**
 - Validates the form.
 - Calls `httpService.AssignClaim()` to submit the claim assignment.
 - Displays success or error messages accordingly.
- 2. getClaims()**
- **Purpose:** Fetches the list of available claims.
- **Variables:**
 - `claimList: any[]` - Stores fetched claims.
 - `errorMessage: string` - Stores an error message if fetching fails.
- **Functionality:**
 - Calls `httpService.getAllClaims()` to retrieve claims.
 - Updates `claimList` with fetched claims.
 - Displays an error message if the request fails.
- 3. getUnderwriter()**
- **Purpose:** Fetches the list of underwriters.
- **Variables:**
 - `underwriterList: any[]` - Stores fetched underwriters.
 - `errorMessage: string` - Stores an error message if fetching fails.
- **Functionality:**
 - Calls `httpService.GetAllUnderwriter()` to retrieve underwriters.
 - Updates `underwriterList` with fetched data.
 - Displays an error message if the request fails.

File: CreateClaimComponent

Variables

- **itemForm:** FormGroup - Manages form controls for creating a claim.
- **formModel:** any - Stores default form values (status initialized as null).
- **showError:** boolean - Flags if an error message should be displayed.
- **errorMessage:** any - Stores the error message text.
- **claimList:** any[] - Holds the list of claims fetched from the backend.
- **assignModel:** any - Placeholder object for assignment data (not used explicitly).
- **showMessage:** any - Flags if a success message should be displayed.
- **responseMessage:** any - Stores success message text.

ConstructorSignature: constructor(

```
public router: Router,
public httpService: HttpService,
private formBuilder: FormBuilder,
private authService: AuthService
```

)

- **Purpose:** Initializes form controls for claim creation.

- **Problem:** Uses `this.formModel.description`, `this.formModel.date`, and `this.formModel.status` as default values, which may be null or undefined.
- **Proposed Solution:** Ensure `formModel` has valid default values before form initialization.

Method Signatures

1. `getClaims()`

`getClaims(): void`

- **Purpose:** Fetches the list of claims for the logged-in policyholder.
- **Functionality:**
 - Retrieves the `userId` from `localStorage`.
 - Calls `httpService.getClaimsByPolicyholder(userId)` to fetch claims.
 - Updates `claimList` with the retrieved data.
 - Displays an error message if the request fails.

2. `onSubmit()`

`onSubmit(): void`

- **Purpose:** Handles the claim creation form submission.
- **Functionality:**
 - Validates the form.
 - Retrieves the `userId` from `localStorage`.
 - Calls `httpService.createClaims(this.itemForm.value, userId)` to submit the claim.
 - Resets the form and refreshes the claim list upon success.
 - Displays an error message if the request fails.

File: `CreateInvestigatorComponent`

Variables:

- `itemForm`: `FormGroup` - Holds the form structure for creating or updating an investigator report.
- `formModel`: `any` - Stores default values for form fields (e.g., status).
- `showError`: `boolean` - Controls the visibility of error messages.
- `errorMessage`: `any` - Stores error messages when an operation fails.
- `assignModel`: `any` - Placeholder object for future assignment logic.
- `showMessage`: `any` - Controls the display of success messages.
- `responseMessage`: `any` - Stores messages related to form submission success or failure.
- `investigationList`: `any[]` - Stores the fetched list of investigations.
- `updateId`: `any` - Holds the ID of an investigation when updating an existing record.

Constructor:

`constructor(router: Router, httpService: HttpService, formBuilder: FormBuilder, authService: AuthService)`

- Initializes the form with fields `report` and `status`, both required.
- Ensures proper form validation before submission.

Methods:

1. `ngOnInit()`

- Calls `getInvestigation()` to load existing investigations on component initialization.
- 2. **`getInvestigation()`**
 - Fetches the list of investigations from the API.
 - Updates `investigationList` with retrieved data.
 - Displays an error message if the request fails.
- 3. **`edit(val: any)`**
 - Prepares the form for editing an existing investigation.
 - Sets `updateId` and populates `itemForm` with the selected record's details.
- 4. **`onSubmit()`**
 - Validates the form before submission.
 - If `updateId` is null, creates a new investigation via `createInvestigation()`.
 - If `updateId` exists, updates the existing investigation via `updateInvestigation()`.
 - Resets the form and refreshes the list after successful submission.
 - Displays an error message if an issue occurs.

File: DashbaordComponent

Variables:

(No class properties are defined in this component.)

Constructor:

- No constructor is defined as this component does not require any dependencies or initialization logic.

Methods:

- This component currently does not contain any methods.

Purpose:

- Acts as the main dashboard component of the application.
- Uses `dashbaord.component.html` for the UI and `dashbaord.component.scss` for styling.
- Can be expanded to include data display, widgets, charts, or navigation controls in the future.

File: LoginComponent

Variables:

- `itemForm`: `FormGroup` – Holds form controls for login fields (username, password).
- `formModel`: `any = {}` – Stores initial form values.
- `showError`: `boolean = false` – Indicates if an error message should be displayed.
- `errorMessage`: `any` – Stores error messages for failed login attempts.

Constructor Signature:

`constructor(public router: Router, public httpService: HttpService, private formBuilder: FormBuilder, private authService: AuthService)`

- Initializes the login form with username and password fields using `FormBuilder`.
- Applies required validators to ensure both fields are filled before submission.

Methods:

1. `ngOnInit(): void`

- Lifecycle hook that runs when the component initializes.
- Currently, it does not perform any actions.

2. **onLogin()**

Purpose: Handles user authentication and login process.

Functionality:

- Validates the form before submission.
- Calls `httpService.Login()` to authenticate user credentials.
- If successful, stores the user's role, token, and user ID using `authService`.
- Navigates to the dashboard and refreshes the page.
- Displays an error message if login fails.

3. **registration()**

Purpose: Redirects users to the registration page.

Functionality:

- Uses `router.navigateByUrl('/registration')` to navigate to the registration screen.

File: RegistrationComponent

Variables:

- `itemForm`: `FormGroup` – Manages the registration form fields.
- `formModel`: `any = {role: null, email: "", password: "", username: ""}` – Stores default form values.
- `showMessage`: `boolean = false` – Controls the visibility of the registration success message.
- `responseMessage`: `any` – Stores the success response message.

Constructor:Signature:

`constructor(public router: Router, private bookService: HttpService, private formBuilder: FormBuilder)`

- Initializes the registration form with fields (email, password, role, username).
- Uses Validators to enforce required fields and validate email format.

Methods:

1. **ngOnInit(): void**

- Lifecycle hook that executes when the component is initialized.
- Currently does not perform any actions.

2. **onRegister()**

Purpose: Handles user registration.

Functionality:

- Validates the form before submission.
- Calls `bookService.registerUser()` to register the user.
- Displays a success message on successful registration.
- Resets the form after completion.
- Marks all fields as touched if validation fails.

File: UpdateClaimComponent

Variables:

- `itemForm`: `FormGroup` – Manages the claim update form fields.
- `formModel`: `any = {status: null}` – Stores default form values.
- `showError`: `boolean = false` – Controls the visibility of error messages.
- `errorMessage`: `any` – Stores error messages for failed operations.
- `claimList`: `any[]` – Holds the list of claims fetched from the server.

- assignModel: any = {} – Reserved for additional assignments (currently unused).
- showMessage: any – Reserved for displaying messages (currently unused).
- responseMessage: any – Stores a response message for success notifications.
- updatedId: any – Stores the ID of the claim being updated.

Constructor Signature:

constructor(public router: Router, public httpService: HttpService, private formBuilder: FormBuilder, private authService: AuthService)

- Initializes the update claim form with fields (description, date, status).
- Uses Validators to enforce required fields.

Methods:

1. **ngOnInit(): void**

- Lifecycle hook that executes when the component is initialized.
- Calls getClaims() to fetch the list of claims.

2. **getClaims()**

Purpose: Fetches all claims from the server.

Functionality:

- Calls httpService.getAllClaims() to retrieve claims.
- Stores the data in claimList.
- Displays an error message if the API call fails.

3. **edit(val: any)**

Purpose: Prepares a claim for updating.

Functionality:

- Stores the selected claim's ID in updatedId.
- Populates the form fields (description, status, date).
- Calls formatDate() to ensure the date format is consistent.

4. **formatDate(date: string | Date): string**

Purpose: Formats the date into YYYY-MM-DD for form compatibility.

Functionality:

- Converts the input date to a Date object.
- Extracts and formats year, month, and day.

5. **onSubmit()**

Purpose: Handles the claim update submission.

Functionality:

- Validates the form before submission.
- Calls httpService.updateClaims() to update the selected claim.
- Resets the form and reloads claims after a successful update.
- Displays an error message if the API call fails.

File: UpdateClaimInvestigatorComponent

This component allows an investigator to update the status of assigned claims.

Variables:

- itemForm: FormGroup – Manages the update form with status field.
- formModel: any = { status: null } – Holds default values for the form.
- showError: boolean = false – Flag to display error messages.

- errorMessage: any – Stores error messages in case of API failure.
- claimList: any[] – Holds the list of claims assigned to the investigator.
- assignModel: any = {} – Reserved for additional assignments (currently unused).
- showMessage: any – Reserved for success messages (currently unused).
- responseMessage: any – Stores response messages.
- updateId: any – Stores the ID of the claim being updated.

Constructor Signature:

```
constructor(
  public router: Router,
  public httpService: HttpService,
  private formBuilder: FormBuilder,
  private authService: AuthService
)
  • Initializes itemForm with status as a required field.
  • Injects Router, HttpService, FormBuilder, and AuthService.
```

Methods:

ngOnInit(): void

- Lifecycle hook executed when the component initializes.
- Calls getClaims() to fetch investigator-assigned claims.

getClaims()

Purpose: Retrieves claims assigned to the logged-in investigator.

Functionality:

- Extracts userId from localStorage.
- Calls httpService.getClaimsByUnderwriter(userId).
- Stores retrieved claims in claimList.
- Handles errors and displays appropriate messages.

edit(val: any)

Purpose: Sets the claim ID for updating.

Functionality:

- Assigns val.id to updateId.

onSubmit()

Purpose: Updates the status of the selected claim.

Functionality:

- Validates the form before submission.
- Calls httpService.updateClaimsStatus(status, updateId).
- Resets the form and reloads claims after a successful update.
- Displays an error message if the update fails.

File: AppComponent

This is the root component of the Angular application. It manages user authentication status, role-based access, and logout functionality.

Variables:

- IsLogin: any = false – Tracks whether the user is logged in.
- roleName: string | null – Stores the role of the logged-in user.

Constructor Signature:

constructor(private authService: AuthService, private router: Router)

- Injects AuthService to handle authentication.
- Injects Router for navigation control.
- Retrieves login status and user role from AuthService.
- Redirects to the /login page if the user is not logged in.

Methods:

logout()

Purpose: Logs out the user and refreshes the page.

Functionality:

- Calls authService.logout() to clear authentication data.
- Reloads the page using window.location.reload() to reset the application state.

File : AuthService

This Angular service manages authentication, including handling JWT tokens, user roles, login status, and logout functionality.

Properties:

- token: string | null = null – Stores the authentication token.
- isLoggedIn: boolean = false – Tracks whether the user is logged in.
- id: string | null | undefined – Holds the user ID (not used in the code).

Methods:

1. saveToken(token: string)

Purpose: Stores the authentication token in local storage and sets login status.

- Saves the token to localStorage for persistence.
- Updates this.token and this.isLoggedIn.

2. SetRole(role: any)

Purpose: Stores the user's role in localStorage.

3. get getRole(): string | null

Purpose: Retrieves the user's role from localStorage.

4. get getLoginStatus(): boolean

Purpose: Checks if a user is logged in by verifying the presence of a token in localStorage.

5. getToken(): string | null

Purpose: Retrieves the stored token from localStorage.

6. logout()

Purpose: Clears authentication data from localStorage and resets login status.

7. saveUserId(userid: string)

Purpose: Stores the user ID in localStorage.

File : HttpService

This Angular service (HttpService) handles HTTP requests related to claims, investigations, user authentication, and role-based operations using JWT tokens for authorization.

Properties:

- **serverName:** string – Stores the base API URL from the environment configuration.

Methods:

1. **getInvestigations()**
Purpose: Fetches a list of investigations for an investigator.
 - Requires a valid JWT token in the request headers.
2. **getClaimsByUnderwriter(id: any)**
Purpose: Retrieves claims assigned to a specific underwriter.
3. **getClaimsByPolicyholder(policyholder: any)**
Purpose: Fetches claims associated with a policyholder.
4. **getAllClaims()**
Purpose: Retrieves all claims available to an adjuster.
5. **GetAllUnderwriter()**
Purpose: Fetches a list of all underwriters.
6. **updateInvestigation(details: any, investigationId: any)**
Purpose: Updates an existing investigation.
7. **createInvestigation(details: any)**
Purpose: Creates a new investigation.
8. **createClaims(details: any, policyholderId: any)**
Purpose: Submits a new claim for a policyholder.
9. **updateClaims(details: any, claimId: any)**
Purpose: Updates an existing claim.
10. **updateClaimsStatus(status: any, claimId: any)**
Purpose: Updates the status of a claim during the review process.
11. **AssignClaim(details: any)**
Purpose: Assigns a claim to an underwriter.
12. **Login(details: any)**
Purpose: Handles user login and returns a JWT token.
13. **registerUser(details: any)**
Purpose: Registers a new user in the system.

Each method appends an authorization token in the request headers for secure API communication.